Prog C++

FS 2026 – Prof. Dr. Christian Werner

Autoren: Fabian Steiner
Mitwirkende: Mirko Ratti

V2.0.20260505 https://github.com/Iceteavanill/Cheatsheet-Prog_Cpp_FS_2024_OST

1 Basics

1.1 Grundsätzlich

C++ ist sehr ähnlich zu C. Es gibt aber einige wichtige Änderungen zu C:

- zu benutzender Compiler heisst jetzt `clang++`
- Nativen Bool Typ : **bool**
- richtiges Konstanten Schlüsselwort : **constexpr**
- Standartbibliotheken haben kein .h mehr beim Aufrufen
- C Standartbibliotheken können weiter verwendet werden, haben aber ein C im Header(stdio.h → cstdio)
- Char-Literale (z. B. 'A') haben in C++ den Datentyp char (in C war es int)(Brauchen ein const)
- Es gibt benannte Namensräume. Wichtigster Namensraum für die STL: `std`

1.2 Hello world C++

```cpp
#include <iostream> // iostream fuer Kommandozeilenausgabe
using namespace std; // dadurch entfaellt das "std::" vor cout
int main() // 'void' can be omitted
{
    cout << "Hello World!" << endl; // "Einschieben" von hello world in den stream
    return 0; // ein return 0 ist in c++ am Ende der main funktion fakultativ
}
```

1.3 Range-based for Loop

Range based for Loops laufen automatisch ein Array ab, ohne das zuerst die Grösse ermittelt werden muss. Die C for loop syntax wurde etwas erweitert. Syntax:

```cpp
for(<DataType> <IteratorName> : arrayName){
    ;//Loop where <IteratorName> is each element
}
```

Siehe: Als Array Name wird ein `Pointer` auf das Array verlangt, nicht auf das erste Element! Ein foreach funktioniert **nicht** mit einem **Pointerpointer**.

1.4 Streams

Ein Stream repräsentiert einen generischen sequentiellen Datenstrom. Z.b: ein Eingabefeld, Dateien oder Netzwerktraffic.
Die wichtigsten Operatoren sind:

- << : Inserter → Daten einfügen
- >> : Extractor → Daten herausholen

Für Standardklassen sind diese Operatoren bereits definiert.

Standardstreams

- **cin** : Standard Eingabe
- **cout** : Standard Ausgabe
- **cerr** : Standard Fehlerausgabe,
- **clog** : Standard Log Ausgabe, möglicherweise mit cerr gekoppelt

`cin/cout` **Immer ganz links!**, operatoren klar und lesbar anordnen

1.4.1 Streamformatierung

Formatierungen der Streams können mit folgenden Schlüsselwörtern erreicht werden:

Flag	Wirkung	Gültigkeit
`boolalpha`	bool-Werte werden textuell ausgegeben	Dauerhaft
`dec`	Ausgabe erfolgt dezimal	Dauerhaft
`fixed`	Gleitkommazahlen im Fixpunktformat	Dauerhaft
`hex`	Ausgabe erfolgt hexadezimal	Dauerhaft
`internal`	Ausgabe innerhalb Feld	Dauerhaft
`left`	linksbündig	Dauerhaft
`oct`	Ausgabe erfolgt oktal	Dauerhaft
`right`	rechtsbündig	Dauerhaft
`scientific`	Gleitkommazahl wissenschaftlich	Dauerhaft
`showbase`	Zahlenbasis wird gezeigt	Dauerhaft
`showpoint`	Dezimalpunkt wird immer ausgegeben	Dauerhaft
`showpos`	Vorzeichen bei positiven Zahlen anzeigen	Dauerhaft
`skipws`	Führende Whitespaces ignorieren	Dauerhaft
`unitbuf`	Leert Buffer dopo ogni operazione	Dauerhaft
`uppercase`	Kleinbuchstaben in Grossbuchstaben wandeln	Dauerhaft
Benötigen Header <iomanip>		
`setw(n)`	Setzt minimale Feldbreite	Einmalig
`setprecision(n)`	Setzt Anzahl Stellen (signifikant o. Nachkomma)	Dauerhaft
`setfill(c)`	Setzt Füllzeichen (Standard: Leerzeichen)	Dauerhaft

1.5 Namespaces

- Namespaces helfen, Namenskonflikte zu verhindern. Sie fügen ein Namensvorsatz zu allen Variablen.
- Per ogni Unit (Modul) dovrebbe essere definito un namespace
- Mit kleine Buchstaben starten
- namespaces senza nome: evitano uso di `static` per var. locali
- L'operatore `::` ritorna sempre la variabile o funz. più globale

```cpp
void f(); // name in global Namespace (1. f())
namespace myLib{void f(); int i} // namespace myLib (2. f())
f(); // f von global Namespace
myLib::f(); // f von namespace myLib
using namespace myLib;
using myLib::i; // posso usare using anche per singole var.
::f(); // 1. f()
f(); // 2. f()

// Nameless namespace
namespace {
    int localCounter = 0; // Invisibile all'esterno di questo file
}
void funzionePubblica() {
    localCounter++; // instead of static
}
```

1.6 Kostanten

- `#define` soll nich mehr verwendet werden
- `constexpr` verwenden!
- `enum` da usare solo se l'automatismo di +1 è utile

1.6.1 Optiemierung `constexpr`

Ambito	Sintassi Consigliata
Header file	`inline constexpr`
Class or function	`static constexpr`

2 Funktionen

2.1 Default Argumente

- **Prototipo**: I valori predefiniti si assegnano **solo** nell'interfaccia.
- **Omissione**: I parametri con default possono essere saltati nella chiamata.
- **Ordine**: I parametri di default devono trovarsi **sempre alla fine (a destra)** della lista parametri.

```cpp
void func(int a = 0, int b = 0, int c = 0){}
func();          // a = 0, b = 0, c = 0
func(1);         // a = 1, b = 0, c = 0
func(1, 1);      // a = 1, b = 1, c = 0
func(1, 1, 1);   // a = 1, b = 1, c = 1
//Alle Funktionsaufrufe valide

void func2(int a, int b = 0, int c = 0); // korrekte Reihenfolge
```

Default Argumente können überall verwendet werden bis auf Aufrufparameter von Operatoroverloading. Dort sind sie **nicht** erlaubt. Dort machen sie aber ohnehin keinen Sinn.

2.2 Funzioni Inline

- **Sostituto Macro**: Alternativa sicura alle macro C.
- **Zero Overhead**: Codice inserito direttamente (nessun salto di funzione).
- **Type Safety**: Il compilatore garantisce il controllo dei tipi.
- **Target**: Funzioni piccole chiamate frequentemente (es. cicli).
- **No-go**: Inapplicabile a ricorsione o puntatori a funzione.
- **Header-only**: Se usato nei file Header, definizione func. direttamente in file.h
- **Ottimizzazione**: Richiede flag attivi (es. `-O3`).

```cpp
// Definizione nell'Header (.h)
inline int quadrato(int x) { return x * x; }

// Compilazione con ottimizzazione
clang++ -O3 main.cpp
```

3 Objektorientierung

Objektorientierung soll es erleichtern, eine Abbildung der Realität zu erstellen. Viele Dinge aus der Wirklichkeit können als Modell dargestellt / simplifiziert werden. Diese Modelle können in Software in sogenannte Objekte `umgewandelt` werden.

3.1 Objekte

Objekte stellen Dinge, Sachverhalte oder Prozesse dar. Sie sind ein rein gedankliches Konzept.
Sie kennzeichnen sich durch:

- eine **Identität**, welche es erlaubt Objekte voneinander zu unterscheiden
- **statische Eigenschaften** zur Darstellung des **Zustandes** des Objekts in Form von **Attributen**
- **dynamische Eigenschaften** zur Darstellung des **Verhaltens** des Objekts in Form von **Methoden**

3.2 Klassen & Instanzen

Ähnliche Objekte können in **Klassen** zusammengefasst werden, um Programmieraufwand tiefer zu halten. Verwendete Objekte werden als **Instanz** eingesetzt. Diese Objekte haben dieselben Attribute, allerdings andere Werte.
Eine Beispielklasse:

```cpp
#include <string>//Stl Strings
class Student {
    public: // folgende Elemente sind Public
        int IDNumber;
        void doStuff(int time = 0);
    private: // folgende Elemente sind Private(standart)
        std::string name;//String
        float bierkonsum;
        unsigned long long int investedHoursInET;
};
// ------ start cpp ------
//implementierung der doStuff Funktion
void Student::doStuff(int time = 0) {
} //ToDo

int main(){
    Student typETStudent;//instanz erzeugen
    typETStudent.IDNumber = 1;//Attribute veraendern
    typETStudent.doStuff();//Methode rufen
}
```

3.2.1 Reihenfolge der Header-Datei (.h)

1. Dateikommentar mit Lizenzvereinbarung.
2. `Includes` *des verwendete System Header <...>*
3. `Includes` *der Projektbezogenen Header "..."*
4. *Definition von Konstanten*
5. *Typedef's und Definitionen von Strukturen*
6. *allenfalls extern-Deklaration von Global-Variablen*
7. *Funktionsprototypen, ink. Kommentar der Schnittstelle, bzw. Klassendeklarationen*

- Die *blauen* Punkte müssen sich im Includeguard befinden(#pragma once).
- Pro Header-Datei sollte nur eine Oberklasse deklariert sein.
- Kleine Unterklassen können im gleichen Header stehen.
- Kein `using namespace ...` in header files

3.2.2 Reihenfolge der Implementation (.cpp)

1. Dateikommentar mit Lizenzvereinbarung
2. `includes` der eigenen Header
3. `includes` der Projektbezogenen Header
4. `includes` der verwendeten System Header
5. allenfalls globale und statische Variablen
6. Präprozessor-Direktiven
7. Funktionsprototypen von lokalen, internen Funktionen (in nameless Namespace)
8. Definition von Funktionen und Klassen (Kommentare nicht wiederholen)

3.3 UML

Die `Unified Modeling Language` ist eine normierte Sprache um Objekte grafisch darzustellen, **programmierspracheunabhängig**.

3.3.1 UML-Klassendiagramm Notation

Besteht immer aus 3 Boxen:

Student
+ IDNumber : int # name : string - bierkonsum : float # investedTimeInET : int
- doStuff(time : int) + *useless()*

- Zu oberst der Name
- In der Mitte Attribute(Variablen)
- Unten Methoden.
- Methoden und Attribute fangen mit einem Symbol an, welches die **Sichtbarkeit** definiert und haben ein bestimmtes Merkmal:
- Parole come `void` o `const` sono specifiche per linguaggio, da non includere

- + : public (überall sichtbar)
- - : private (nur in aktueller Klasse sichtbar)
- # : protected (in aktueller und Unterklassen sichtbar)
- *Italic* : Funktion oder Klasse ist Abstrakt (=0)
- Unterstrichen : Funktion oder Attribut ist static
- `<Name der Klasse>` : Konstruktor (ohne <>)
- `~<Name der Klasse>` : Destruktor (ohne <>)

3.4 class vs struct

Eine class und struct können fast identisch verwendet werden. Eine class hat **standardmässig Sichtbarkeit private** sonst muss explizit mit `public:` gekennzeichnet, Struct public(wenn nichts definiert wird).

3.4.1 Zugriffsschutz

Modificatore	Descrizione e Best Practice
`public`	Accessibili dall'interno e dall'esterno della classe. → Quasi tutti i **metodi** sono pubblici. → **attributi** non dovrebbero *mai* essere pubblici.
`protected`	Accessibili dall'interno della classe e dalle **classi derivate**. → Da utilizzare con parsimonia.
`private`	Accessibili **solo** all'interno della classe stessa. → Per tutti gli attributi e per metodi locali specifici.

3.5 Inkludierte Dokumentation

Mann kann im H-File direkt die Dokumentation zu der Schnittstelle schreiben. Das ermöglicht das einfachere Verwenden der Schnittstelle.

```cpp
/**
 * @brief Kurzzusammefassung der Klasse
 *
 */
class stuff{
public:
    /**
     * @brief Funktion von x
     *
     */
    double x;
    /**
     *
     * @brief Kurzzusammefassung der Funktion
     *
     * @param eingang Funktion des Parameter
     * @return nix, aber falls...
     *
     */
    void doStuff(stuff eingang);
};
```

Es kann immer mehr geschrieben werden, man sollte sich aber auf das Nötige beschränken.

4 Konstruktoren und Destruktoren

I costruttori (CTOR) e i distruttori (DTOR) sono metodi speciali che gestiscono il ciclo di vita di un oggetto. Entrambi hanno lo stesso nome della classe e **non** hanno alcun tipo di ritorno (nemmeno `void`).

4.1 Konstruktoren (CTOR)

Il compito principale del CTOR è l'inizializzazione "pulita"di tutti gli attributi della classe.

- **Default-Konstruktor:** Non ha parametri. Viene invocato automaticamente se l'oggetto è dichiarato senza argomenti (es. `Stack s;`). Se non definito, il compilatore ne crea uno implícito (che però non inizializza i tipi primitivi).
- **Copy-Konstruktor:** Riceve una `const`-Referenz alla propria classe (`TString( const TString& s)`). Viene usato per creare un oggetto come copia di uno esistente.
- **Explicit-Konstruktor:** Marcando un CTOR a parametro singolo con `explicit`, si impediscono conversioni implicite indesiderate (es. `TString s = 5;`).

Werden verschiedene Konstruktoren definiert, muss der Default zuerst aufgerufen werden, wenn dieser erhalten bleiben soll.

4.1.1 Best Practices di Inizializzazione

Una delle responsabilità primarie del costruttore è garantire che l'oggetto nasca in uno stato valido. Ciò implica che **tutti** gli attributi della classe debbano essere inizializzati su un valore definito.

- **Assegnamento nel corpo (4.4):** Meno efficiente, i membri vengono prima creati con valori predefiniti e poi sovrascritti.
- **Initialisierungsliste (5.3):** Il metodo preferito per motivi di efficienza; gli attributi vengono inizializzati direttamente al momento della creazione.
- **Member In-Class Initialization:** (Da C++11) Permette di impostare valori di default direttamente nella dichiarazione della classe, semplificando i costruttori ed evitando dimenticanze.

4.2 Gestione Memoria: Shallow vs. Deep Copy

Quando una classe gestisce puntatori a memoria allocata sul Heap (`new`), il comportamento di default è critico:

Shallow Copy (Default): Il compilatore copia solo l'indirizzo del puntatore. Entrambi gli oggetti puntano alla stessa memoria. Se uno la libera, l'altro punta a memoria non valida (*Double Free* o *Segmentation Fault*).

Deep Copy (Manuale): **È necessario** definire un Copy-Konstruktor proprio che allochi nuovo spazio sul Heap e copi i dati effettivi.

4.3 Destruktor

Ein Destruktor eintfernt eine Instanz aus dem Speicher, hat keine Aufrufparameter und auch kein Rückgabewert. Es kann immer nur **einen** Destruktor geben. Wenn der Destruktor fertig ist, sollte kein Speicher mehr vom Objekt belegt sein.
Destruktoren sollten immer **virtuell definiert werden** (siehe 10.2.1). Mit anderen virtuellen Funktionen **müssen** sie virtuel definiert werden. Der Funktionsname beginnt mit einer Tilde(~). Der Destruktor wird evtl. auch Implizit aufgerufen z.b., wenn aus einer Funktion wieder herausgesprungen wird.

4.4 Beispiel

```cpp
// START H FILE
class Storage{
    public:
        Storage();
        virtual ~Storage();
        void add(int in);
```

```cpp
        void nix(Storage inC);
    protected:
        ...
    private:
        int* data;
        int size;
};//eine Klasse die Int Werte speichert, aehnlich wie ein array

// END H FILE
// START Storage.cpp
Storage::Storage(){ // Konstruktor
    data = nullptr;
    size = 0;
}

Storage::~Storage(){ // Destruktor
    delete[] data;
    data = nullptr;
    size = 0; // Speicher der allokiert wurde sollte hier freigegeben werden
}
void Storage::add(int in){
    // todo
}
void Storage::nix(Storage inC){
    // todo
}

// END Storage.cpp
// START main.cpp

int main(){

    Storage* i1 = new Storage; // default konstruktor
    Storage i2; // default konstruktor
    i1->nix(i2); // Call-by-Value. Eine Kopie von i2 wird auf den Stack gelegt -> Copy-Konstruktor!

    delete i1; // destruktor von i1 wird explizit aufgerufen
    return; // destruktor von i2 wird implizit aufgerufen
}
```

5 Initialisierungslisten und direkte Initialisierung

Die Initialisierung von Datentypen kann auf verschiedene Arten erreicht werden. Es wird zwischen Zuweisung und Initialisierung unterschieden.

5.1 POD's

`plain old datastructure` / built in datatypes

```cpp
// Zuweisung fuer POD:
int i; // Speicher fuer Instanz wird alloziert, Wert ist uninitialisiert
i = 5; // Wert (aus anderer Speicherstelle) wird zugewiesen

// Initialisierung fuer POD: Kein weitere Speicherverbrauch
int i = 5;  // (C-Style)
int i(5); // (C++-Style bis C++11)
int i{5}; // (neuer C++-Style)
// Aus Effizienzgruenden bevorzugen wir (fast) immer die (direkte) Initialisierung!
```

5.2 Non PODs

```cpp
Aclass m;//kein Initialwert
// Vorsicht: Falls "Aclass" gross ist, muss viel kopiert werden
m = otherInstance;

// Copy-Initialisierung fuer non-POD:
Aclass m = otherInstance; //(copy ctor)
Aclass m(otherInstance); // C++-Schreibweise
Aclass m{otherInstance}; // C++-Schreibweise seit c++11

//Besser: direkte Initialisierung auch fuer non-PODs:
Aclass m("Eagle 1", 1, 2, ...); // (vor C++11)
Aclass m{"Eagle 1", 1, 2, ...}; // bevorzugte Schreibw. seit C++11
// Die Instanz wird ohne Umwege mit den gewuenschten Werten in den Speicher geschrieben, sofern ein geeigneter user-defined ctor vorhanden ist!
```

5.3 User defined Constructor

Um eine Klasse richtig initialisieren zu können muss der Ctor korrekt definiert werden. Eine sogenannte **Initialisierungsliste** wird verwendet:

```cpp
student::student(int _semester, long int _eltVerzweiflung):
    semester{_semester},
    eltVerzweiflung{_eltVerzweiflung}
//die Reihenfolge der Initialisierung ist durch die Reihenfolge im Speicher gegeben!
{
    //Rumpf kann leer bleiben Initialisierung bereits fertig
}
```

6 Assertions

Assertions sollten Annahmen über korrekte Wertebelegung darstellen.
Sie sind kein C spezifisches Konzept und sollten **nur zu Testzwecken** eingesetzt werden. Im Releasebuild sollten sie deaktiviert werden. Dafür gibt es spezifische Präprozessorflags. Der Header <**assert.h**> bzw. <**cassert**> stellt hierfür ein Präprozessor-Makro `assert(Bedingung)` zur Verfügung, um solche Zusicherungen auszudrücken.
Beispiel:

```cpp
//#define NDEBUG // Deaktiviert Assertions im Projekt
#include <cassert>
bool testStuff(int* in1){

    assert(in1 != nullptr);
    //assertion

    ;//restliche implementation einer Funktion

}
```

Das obige Beispiel hat eine Assertion. Diese würde im Test, falls ein Nullpointer übergeben wird ein Fehler ausgegeben.

7 This-Pointer

Mit dem `this Pointer` kann ein Pointer des aufrufenden Objekt zurückgegeben werden. Bsp:

```cpp
class Stuff{
    public:
        Stuff& funktion(int valIn);
    private:
        int value;
};
Stuff& Stuff::funktion(int valIn){
    //do Stuff with val z.b. addieren auf interne Variabel
    this->funktion(0);// auch moeglich
    return *this;//this pointer -> wird dereferenziert und Referenz zurueckgegeben
}
int main(){
    Stuff memb;
    memb.funktion(1).funktion(2).funktion(3);
}
```

Nacheinander werden diese Funktionen durchlaufen da Funktion(2) vom zurückgegebenen Pointer aus Funktionsaufruf 1 aus gerufen wird. Die funktion wird **Kaskadiert**

8 Overloading

Operator Overloading bedeutet das bestehende Operatoren `überladen` werden im Sinne neu definiert / darüber geladen. Sie erhalten **neue Bedeutung** für eine Klasse.

8.1 Methodenoverloading

- **Definizione**: Stesso nome per funzioni con parametri diversi (tipo, numero o ordine).
- **Firma**: Composta da *Nome* + *Parametri*. Il tipo di ritorno è **escluso**.
- **Regola**: Se differiscono solo per il tipo di ritorno, il compilatore genera un errore.
- **Best Practice**: Usare l'overloading solo per operazioni comparabili e **non mescolarlo mai con i parametri di default.**

Caratteristica	Overloading	Parametri Default
Memoria	Più funzioni	Singola funzione
Manutenzione	Maggiore	Minore
Flessibilità	Tipi diversi	Valori predefiniti

```cpp
// Approccio Overloading (3 implementazioni)
void print(int i);
void print(char c);
void print(int* arr, int n);
// A dipendena del parametro passato verra' chiamata una div. implementazione di print()
```

8.2 Operator overloading

Es können die inkludierten Operatoren von C++ überladen werden.
Erlaubte Operatoren sind:

+	-	*	/	%	^	&	|	~
!	=	<	>	+=	-=	*=	/=	%=
^=	&=	|=	<<	>>	<<=	>>=	==	!=
<=	>=	&&	||	++	--	,	->*	->
( )	[ ]	new	delete	new[ ]	delete[ ]			

Nicht erlaubte Operatoren sind: '`.` `.*` `::` `?:` `sizeof` `typeid`'.
Es wird zwischen Overloading als *globalen Funktion* und als *Klassen-Methode* unterschieden.

8.2.1 Als Methode (Member function)

Ein Operator kann als Methode einer Klasse definiert werden. Dies ist der Standardfall, wenn der **linke Operand** ein Objekt der eigenen Klasse ist.
Der linke Operand (Objekt selbst) ist dabei implizit, weshalb die Methode **einen Parameter weniger** annimmt als die globale Variante.
Syntax: `<returntype> operator<typ>(type in);`

- **Visibilità a livello di Classe:** In C++, il controllo degli accessi (`private`) agisce a livello di **classe**, non di singolo oggetto.
- **Accesso Incrociato:** Un metodo della classe *A* ha il permesso di accedere ai membri privati di *qualsiasi* istanza di tipo *A*.
- **Applicazioni:** Questa caratteristica è indispensabile per implementare correttamente l'overloading degli operatori (es. `operator+`), i costruttori di copia e i metodi di confronto, dove un oggetto deve leggere i dati interni di un suo 'simile'.

```
// --- H file ---
class Dozent {
private:
    int number;

public:
    Dozent(int n = 0) : number(n) {}

    // 1. Metodo membro: Dozent + Dozent
    Dozent operator+(const Dozent& other) const;

    // 2. Metodo membro: Dozent + int
    Dozent operator+(int val) const;
};

// --- CPP file ---
Dozent Dozent::operator+(const Dozent& other) const {
    return Dozent(this->number + other.number);
}

Dozent Dozent::operator+(int val) const {
    return Dozent(this->number + val);
}
```

Zwingend als Elementfunktion zu implementieren

- Zuweisung (=, +=, etc.)
- Index `[]`
- Funktionsaufruf `()`
- Zeigeroperator `->`

8.2.2 Als globale Funktion (Global)

Globales Overloading wird **zwingend benötigt**, wenn der **linke Operand** *nicht* von der eigenen Klasse ist. Typische Beispiele sind:

- Ein primitiver Datentyp links (z.B. `string + TString`).
- I/O Stream-Operatoren wie <<, bei denen der linke Operand ein Stream-Objekt ist (z.B. `std::cout << obj`).

Syntax: `<returntype> operator<type>(type val1, type val2);`

Friend Keyword:
Oft müssen globale Operatorfunktionen auf private Daten der Klasse zugreifen. In diesem Fall werden sie innerhalb der Klasse mit dem Keyword `friend` deklariert. Dadurch erhalten sie als globale Funktionen Zugriff auf **alle privaten Attribute und Methoden** der Klasse, ohne selbst Teil der Klasse (Methode) zu sein.
Achtung: Friend-Funktionen sollten gezielt eingesetzt werden.

```
 // --- H file ---
class Dozent {
    // La funzione amica permette l'accesso ai membri privati
    // Funzione globale
    friend Dozent operator+(const Dozent& d1, const Dozent& d2
     );

private:
    int numb;

public:
    Dozent(int n) : numb(n) {}
};

// --- CPP file ---
Dozent operator+(const Dozent& d1, const Dozent& d2) {
    return Dozent(d1.numb + d2.numb);
}
```

Beispiel I/O Streams (<<):
Damit Verkettungen wie `cout << a << b;` funktionieren, muss als Rückgabewert eine Referenz auf den `ostream` geliefert werden (`return os;`).

```
friend ostream& operator<<(ostream& os, const TString& s) {
  os << s.getString(); // oder direkter Zugriff auf Attribute
  return os;
}
```

8.2.3 Operatoren und Typumwandlungen

- Per evitare di dover definire innumerevoli varianti di un operatore per ogni combinazione di tipi, è possibile sfruttare le **conversioni di tipo definite dall'utente**.
- Singolo operatore può gestire diversi input se esiste un costruttore in grado di convertire tali tipi nella classe di destinazione.

```
// Dichiarazioni che possono essere sostituite
bool TString::operator<(const TString& s) const;
bool TString::operator<(const char* p) const;
friend bool operator<(const char* p, const TString& s);

// Possono essere ridotte a:

// Queste dichiarazioni sono sufficienti
TString(const char* p); // Typumwandlungs-Ctor
friend bool operator<(const TString& s1, const TString& s2);
```

9 Getter & Settermethoden

Getter und Settermethoden werden Typischerweise verwendet um Lese und Schreibzugriffe auf eine Klasse zu regeln.
Attribute sind **alle Parameter einer Klasse**. Mit einer Get oder Set Methode kann der Zugriff auf diese kontrolliert / angepasst werden.

`const` after the funtion name declares that the method cannot modify any Attribute
immer fur Selector (Getter) benutzen

```
#include <string>
class Stuff{
    public:
       const std::string& getName() const;
    protected://erlaubt schreibenden Zugriff von Unterklassen
       void setName(const std::string& in);
    private:
       std::string name;//privates Attribut
};
const std::string& Stuff::getName() const{ // Getter
    return name;
}
void Stuff::setName(const std::string& in){ // Setter
    name = in;
}
```

9.0.1 Mutable

Si può permettere a determinati attributi di essere modificati da metodi `const` con la keyword `mutable`, **usare il meno possibile**

```
...
mutable int error;
...
```

9.0.2 Const correctness

- **Sola Lettura:** Un oggetto dichiarato come `const` può invocare *solo* metodi dichiarati `const`.
- **Sicurezza:** Se un metodo non è marcato `const`, il compilatore assume che possa modificare l'oggetto e ne impedisce l'uso su istanze costanti per evitare effetti collaterali indesiderati.

Beispiel:

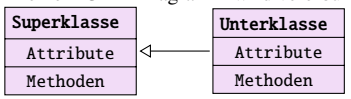
```
class Stack {
public:
    int pop();                // Modifica lo stato (rimuove
     elemento)
    bool isEmpty() const;     // Sola lettura (osserva lo stato
     )
};

void check(const Stack& s) {
    s.isEmpty();  // OK: il metodo promette di non modificare
     s
    // s.pop();   // ERRORE: pop() potrebbe modificare s
}
```

10 Vererbung

Vererbung erlaubt es Attribute und Methoden von anderen Klassen zu übernehmen. Die Oberklasse, Baisklasse oder Superklasse `vererbt` an eine Unterklasse, derived class oder eine Spezialisierung. Bei der Vererbung werden immer **alle** Attribute oder Methoden weitergegeben.

10.0.1 UML

In einem UML Diagramm wird vererbung wie folgt dargestellt:



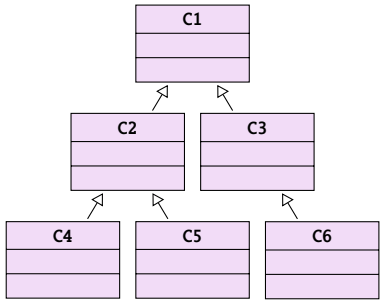
Unterklasse erbt (die Attribute und Methoden) von Superklasse.

Dies stellt eine `ist ein beziehung` dar. Es ist sehr wichtig, dass die Pfeilspitze **genau so!** aussieht wie in der Darstellung da ansonsten Verwechslungsgefahr mit anderen Mechanismen entstehen könnte.

10.0.2 Syntax

```
class Unterklasse : public  Superklasse{
    ...
} ; // Unterklasse erbt Superklasse
```

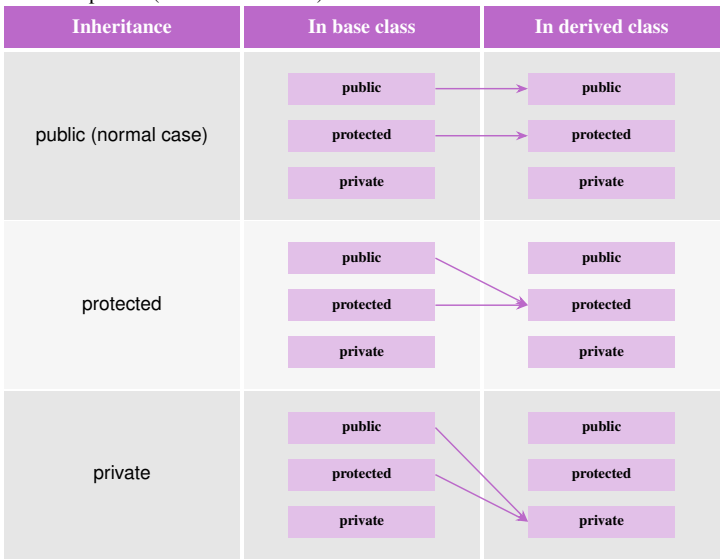
10.0.3 UML++



Vererbung ist auch über mehrere Stufen Möglich:
Z.b. C4 Erbt alles von C2 und C1

10.0.4 Sichtbarkeit

Die Sichtbarkeit ist durch das Public definiert. Es ist möglich die Sichtbarkeit aller Attribute und Methoden einzuschränken. Mit **Public** wird alles mit der originalen Sichtbarkeit übernommen. Mit **protected** wird alles was public war protected. Mit **private** wird alles private (nicht sehr nützlich).



Wird eine Vererbung private durchgeführt, sind keine Attribute oder Methoden mehr sichtbar.
Attribute soll nie `protected` sein

10.0.5 Constructor / Destructor chaining

Logica dei Costruttori (CTOR)

- **Principio di Responsabilità:** Il costruttore di una classe inizializza esplicitamente **solo i propri attributi**.
- **Delegazione:** Per gli attributi della classe base, viene invocato il costruttore della classe base stessa (la base 'si occupa delle proprie faccende').
- **Regola di Priorità:** Gli elementi della classe base devono essere **sempre inizializzati per primi**.

```
Unterklasse::unterklasse(int _initvalOberklasse,
                         int _initvallUnterklasse):
    Oberklasse{_initvalOberklasse},// muss zuerst stehen da
     diese zuerst initialisiert werden muss
    unterklassenAttribut{_initvallUnterklasse}
{} // bleibt leer
```

Logica dei Distruttori (DTOR)

- **Ordine Inverso:** I distruttori vengono eseguiti nell'ordine esattamente opposto ai costruttori.
- **Ordine di Esecuzione:** Dal basso verso l'alto (Sottoclasse → Classe Base).

10.1 Überschreiben von Methoden

Geerbte Funktionen können `überschrieben` werden. Die Unterklasse definiert eine neue Methode mit demselben Namen. Allerdings kann es dann zu Mehrdeutigkeit kommen:
Im folgenden Beispiel haben eine Subklasse und eine Oberklasse eine print Funktion welche **nicht** virtual gekennzeichnet ist:

```
int main() {
    Subclass p;             // Klassenerstellung
    p.print();              // print von Subclass, ok
    Subclass* sPtr = &p;    // Subclass Pointer
    sPtr->print();          // print von Subclass, ok
    Topclass* tPtr = &p;    // !! Topclass Pointer !!
    tPtr->print();          // print von Topclass!!!!!!
    Subclass& sRef = p;     // Subclass ref
    sRef.print();           // print von Subclass, ok
    Topclass& tRef = p;     // !! Topclass ref !!
    tRef.print();           // print von Topclass!!!!!!
}
```

Dieses Beispiel soll zeigen, dass Pointer, welche eigentlich von einer Oberklasse auf eine Unterklasse zeigen auf die `Eigenen` Funktionen der Oberklasse zeigt, solange diese nicht überschrieben worden sind. Dies kann als Verhalten gewünscht sein.

10.2 virtual, final, override und scopeoperator

10.2.1 virtual

- **Zweck:** Kennzeichnet Methoden, die in Unterklassen überschrieben werden sollen. Dies ermöglicht **Dynamic Binding**, sodass zur Laufzeit immer die spezialisierte Methode der Unterklasse aufgerufen wird.
- **Polymorphie:** Eine Klasse, die mindestens eine virtuelle Funktion besitzt, wird als **polymorph** bezeichnet.
- **Vererbung:** Einmal als `virtual` deklariert, bleibt die Methode in der gesamten Vererbungskette virtuell.
- **Wichtige Syntax-Regel:** Das Schlüsselwort `virtual` darf **nur bei der Deklaration** (im H-File) verwendet werden.

10.2.2 Virtual beim Dtor

Ist eine Methode als virtual deklariert, so muss der Dtor **ZWINGEND** auch als virtual deklariert werden.

10.2.3 Override

Soll eine Methode eine geerbte Methode ersetzen so muss diese mit `Override` gekennzeichnet werden. Solch eine Methode ist automatisch auch virtual. Der Compiler kann so überprüfen, ob eine virtual Methode vorhanden ist.

10.2.4 Final

Final kann zum einen verbieten, dass eine Methode einer Klasse überschrieben wird (Logischerweise geht das nur bei virtuellen Methoden). Zum anderen kann sie auch das weitere Erben einer Klasse verbieten.

10.2.5 Scopeoperator

Per Scopeoperator kann (::) eine Methode aus einer Oberklasse gezielt aufgerufen werden. Mit diesem wird garantiert die Funktion welche definiert wird aufgerufen, egal ob diese aufgrund virtual überschrieben wird.

```
class Subexample : public Uppclass{
    virtual ~Subexample(); // Correct Dtor
    virtual void newMethod(); //gets overriden when inherited
    // void oldVirtualMethod(); this inherited func doesnt get
      changed, it stays virtual
    void getsOverridden() override;// overrides method in
     uppclass
    virtual void iAmPerfect() final;// cant be overridden
};
void Subexample::newMethod(){
    Uppclass::aMethodfromUpclass();
    //calls a class from a class higher in the chain
}
```

10.3 Classi Polimorfe e Binding

- **Definizione:** Una classe è polimorfa se dichiara almeno una funzione `virtual`.
- **Tipi di Dato:**
  - **Statischer Datentyp (Tipo Statico):** Il tipo definito alla dichiarazione (tipo del puntatore o della referenza).
  - **Dynamischer Datentyp (Tipo Dinamico):** Il tipo effettivo dell'oggetto a runtime (l'istanza reale).
- **Static Binding (Early Binding):** Avviene durante la compilazione. È lo standard per funzioni non-virtuali e non comporta overhead.
- **Dynamic Binding (Late Binding):** La scelta della funzione avviene a runtime in base al tipo dinamico. Funziona solo tramite puntatori o referenze e comporta un leggero overhead gestito tramite V-Table.

10.3.1 Best Practice e Regole

- **Distruttori:** Devono essere obbligatoriamente `virtual` nelle classi base per garantire la corretta deallocazione delle sottoclassi, specialmente se gestite tramite puntatori alla classe base.
- **Costruttori:** Non possono mai essere dichiarati `virtual`.
- **Metodi:** Dichiarare `virtual` solo se è prevista la sovrascrittura. I metodi non-virtuali non devono essere sovrascritti nelle classi derivate.
- **Override:** Nelle sottoclassi, i metodi sovrascritti devono essere contrassegnati esplicitamente con la parola chiave `override`.

10.3.2 RTTI (Run-time Type Information)

L'RTTI permette di identificare il tipo di un oggetto durante l'esecuzione. Si utilizza la funzione `typeid()` inclusa nell'header `<typeinfo>`, che restituisce un oggetto di tipo `std::type_info`.

```
#include <typeinfo>
int main() {
    int a{0};
    float b{0.0f};
    if (typeid(a) != typeid(b)) {// is always true
    }// because they have a different type
}
```

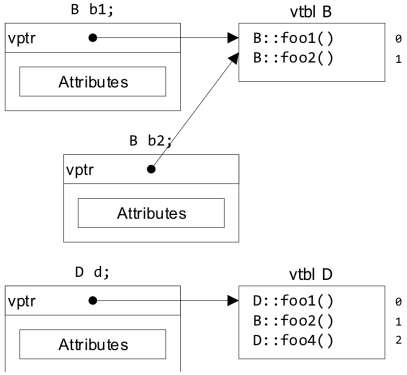
10.3.3 Implementazione: La V-Table

L'implementazione tecnica del polimorfismo e del RTTI si basa sulla **V-Table**:

- Ogni classe polimorfa possiede una V-Table contenente i puntatori alle sue funzioni virtuali.
- Ogni istanza della classe contiene un puntatore nascosto (**V-pointer**) che punta alla V-Table della propria classe.
- A runtime, l'istanza utilizza il V-pointer per risolvere l'indirizzo della funzione corretta.

Se un array di puntatori alla classe base contiene oggetti di diverse sottoclassi, ogni oggetto avrà un V-pointer che punta alla V-Table specifica della propria classe, garantendo che vengano chiamate le versioni corrette dei metodi sovrascritti.

```
class B {
  public:
    virtual void foo1();
    virtual void foo2();
    void foo3();
  };

class D: public B {
  public:
    virtual foo1()
     override;
    virtual void foo4();
  };
```



10.4 Abstrakte Klassen

Wenn Klassen zwar festlegt, dass eine Methode da ist, diese aber noch nicht implementieren, nennt man das eine abstrakte Methode. Man spricht dann von einer abstrakten Klasse. Eine abstrakte Methode wird in UML ***kursiv*** dargestellt. Es ist egal, ob die abstrakten Methoden selbst definiert sind oder geerbt. Abstrakte Klassen kann man nicht instanziieren. Es gibt kein spezielles Schlüsselwort dafür, man weist der Methode bei Definition 0 zu:

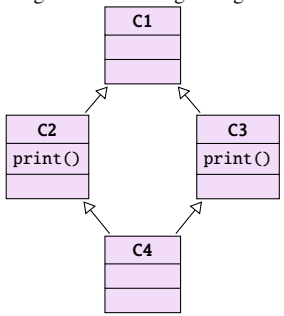
```
class ModernArt{// abstrakte klasse (wegen abstrakter Methode)
    virtual void aMethod() = 0;// abstrakte Methode
};
```

10.5 Organisation der H files

Generell gilt immer noch: Jedes H File hat ein Cpp File. Allerdings können Unterklassen in ein H File zusammengefasst werden, falls diese nicht allzu Umfänglich sind. Ansonsten sollte ein neues H File erstellt werden. Für abstrakte Klassen fällt die Implementierung in einem Cpp file weg.

10.6 Mehrfachvererbung

Folgender Vererbungszweig ist möglich:



Solche Gebilde sind zwar möglich, sollten jedoch vermieden werden, da schnell sehr kompliziert und kaum lesbar. Es entsteht schnell Mehrdeutigkeit durch die verschiedenen Erbzweige.

Se nelle 2 classi parenti è definito un **metodo con lo stesso nome**, è necessario ridefinirlo nella sottoclasse definendo il giusto comportamento (ad es. `C1::print()`)

Bei der Definition einer Klasse können weitere Oberklassen mit einem Komma getrennt angegeben werden

```
class Unterklasse : public Oberklasse1, public Oberklasse2{};
```

10.6.1 Diamond Problem

- **Il Problema**: Senza accorgimenti, `C4` conterrebbe due copie separate di `C1` (tramite `C2` e tramite `C3`). Generando ambiguità nell'accesso ai membri.
- **La Soluzione** (***Virtual Inheritance***):la parola chiave `virtual` nelle classi intermedie (`C2` e `C3`) istruisce il compilatore a creare un'unica istanza condivisa di `C1`.
- **Ordine di Inizializzazione**: Segue una priorità gerarchica:
  1. **Base Virtuale**: `C1` viene inizializzata per prima.
  2. **Basi Non-Virtuali**: `C2` e poi `C3` (seguendo l'ordine di dichiarazione in `C4`).
  3. **Classe Derivata**: Il corpo del costruttore di `C4`.

```
class C1 { ... };

// Il virtual va inserito qui (classi intermedie)
class C2 : virtual public C1 { ... };
class C3 : virtual public C1 { ... };

// C4 eredita normalmente; ordine ctor: C1 -> C2 -> C3 -> C4
class C4 : public C2, public C3 {
public:
    C4() : C1(), C2(), C3() { ... }
};
```

10.7 Static

Static im Zusammenhang mit Klassen bindet eine Methode oder ein Attribut an die Objektklasse und nicht an die Objektinstanz (ähnlich zu Python Klassenvariablen).

10.7.1 Methoden

Static Methoden müssen im Header-File deklariert und im Cpp-File definiert werden. Sie dürfen nur auf Static definierte Attribute zugreifen da **keine Objektinstanz vorhanden.** Hauptanwendungen sind Hilfsfunktionen wie Funktionsaufrufcounter oder Umrechnungsfunktionen. Bei der Deklaration wird der Klassennamen verwendet (siehe Bsp).

10.7.2 Attribute

Static Attribute müssen im H deklariert **und im Cpp File** definiert werden. Sie werden verwendet für z.B. Objektspezifische Konstanten oder im Zusammenhang mit Static Counter. (*class scope*)

10.7.3 Bsp

```
// Logger.h
class Logger {
    public:
        static void log(const std::string& message);
    private:
        static int nrObjects; // Non inizializzabile dirett.
};
// Logger.cpp
int Logger::nrObjects = 34; // Necesario!
// main.cpp
int main() {
    // Benutze log aus Logger, ohne Instanz:
    Logger::log("C++ ist super!");// class name wird verwendet
     !
    return 0;
}
```

11 Memorymanagement

Es gibt verschiedene Gründe warum nicht immer gleich viel Speicher verwendet wird z.B. da nicht unbegrenzt vorhanden ist. Speicher wird nur dann verwendet, wenn er wirklich gebraucht wird. Das Memorymanagement muss aktiv beachtet werden. Es wird zwischen automatischen und manuellen Speichermanagement unterschieden.

11.1 Automatisch

Automatisches Speichermanagement wird anhand der Lebenszeit einer Variable festgestellt und während dem Programmieren bereits festgelegt. Der Speicher liegt auf dem sogenannten **Stapel / Stack**.
Wird eine Funktion aufgerufen, werden die benötigten Variablen auf dem Stack definiert. Ebenso wird eine Returnadresse zur Funktion, die die Funktion aufgerufen hat abgelegt. Wie die Daten angeordnet werden, ist Architektur-abhängig. Wird die Funktion wieder verlassen, werden die Variablen vom Stack wieder entfernt und zur aufrufenden Funktion zurückgesprungen.

11.2 Manuell/dynamisch

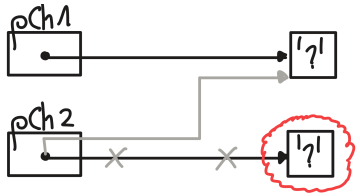
Man kann auch manuell Speicher anfordern. Zum Beispiel, falls zur Compilezeit nicht bekannt ist, wie viel Speicher gebraucht wird. Dieser Speicher ist auf dem sogenannten **Haufen / Heap**.
Der Heap ist ein Speicherbereich, der vom Betriebssystem bereitgestellt und verwaltet wird.
Zur Laufzeit wird Speicher dynamisch angefordert. Dieser bleibt so lange belegt bis dieser Manuell wieder freigegeben wird.
Es kann aber auch sein, falls der Speicher stark fragmentiert ist, das kein Speicher bereitgestellt werden kann. Generell ist bei sehr vollem Speicher das Arbeiten erschwert.

11.2.1 Memory leaks

Falls ein Programm unkontrolliert Speicher aufnimmt oder den ihm zur Verfügung gestellte nicht wieder freigibt, spricht man von einem Speicherleck. Diese sind zu verhindern da diese zu einer Systemüberlastung und Abstürzen führen.

Beispiel:

```
char* pCh1 = new char;
char* pCh2 = new char;

std::cin >> *pCh1;

pCh2 = pCh1;
// pCh2 points now to the Ch1
    memory cell
// The access to the memory
    cell of pCh2
// is now lost (memory leak!)

delete pCh1;
delete pCh2;
// ERROR, already with pCh1
    freed
```



11.3 Speichermanagement funktionen

Es gibt einige Funktionen für Speichermanagement. In C und C++ sind diese leicht verschieden. Generell geben diese einen Pointer zurück, welcher auf freien Speicher zeigt. **Diese müssen immer auf einen Nullpointer geprüft werden, da keine Garantie vorhanden ist, ob überhaupt Speicher vorhanden ist!** Generell sollte mit solchen Pointer immer misstrauisch gearbeitet werden und nach Abschluss immer wieder freigegeben werden.

11.3.1 C: `malloc()`, `calloc()`, `free()`

malloc() gibt einen Voidpointer(pointercasting nicht vergessen) zurück welche auf eine angeforderte Grösse an Bytes Speicher zeigt. Der Speicher ist **nicht** initialisiert
calloc() macht dasselbe, initialisiert aber den Speicher auf 0.
free() gibt den Speicherbereich wieder frei.

```
#include <stdio.h>
```

```c
#include <stdlib.h>

void *malloc(size_t size);
// Alokiert Speicherbereich size in Bytes
void *calloc(size_t size);
// Alokiert Speicherbereich size in Bytes und setzt diesen 0
free(void *ptr);
// Gibt Speicherbereich auf welcher ptr zeigt wieder frei

int main(){
    int* iPtr = NULL;
    iPtr = (int*)malloc(3*sizeof(int));
    // Speicher fuer 3 Ints reservieren
    // Insert Nullpointer check here!!!
    free(iPtr);
}   // Speicher wider freigegen
```

Falls kein Speicher allokiert werden kann, wird ein NULL-pointer zurückgegeben.

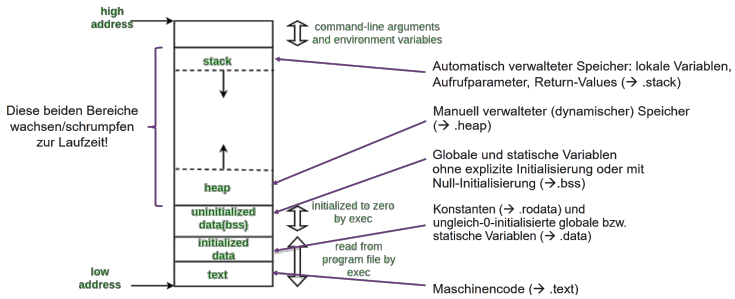
11.3.2 C++: new, delete, new[], delete[] (Operatoren)

Der **new**-Operator erstellt ein Pointer welcher auf die mitgegebene Grösse zeigt.
new[ ] macht dasselbe, ausser das dieser ein Array zurückgibt.
delete / delete[ ] gibt den Speicher wieder frei. Dieser kann auch auf den Nullpointer ausgeführt werden. Dieser ist `Delete` egal.

```c
int main(){
    int* intPtr = new int;
    // Allokiert fuer intPtr ein Speicherbereich eines Ints
    int* intArrayPtr = new int[10];
    // Allokiert fuer intArrayPtr ein Speicherbereich eines
     Int Array der groesse 10
    if(intPtr == nullptr) return -1;// Nullpointercheck

    delete intPtr; // Gibt intPtr wieder frei
    delete[] intArrayPtr; // Gibt intArrayPtr wieder frei
    intArrayPtr = intPtr = nullptr
}
```

11.4 Speicheraufbau

Der Speicheraufbau bildlich dargestellt sieht ungefähr so aus:



Je nach Architektur kann es auch sein, dass die hohen und tiefen Adressen anders herum sind.

11.5 Referenzen

Una referenza è un **alias** (un nome alternativo) per una variabile o una cella di memoria già esistente. Rappresentano una versione più moderna, sicura e ristretta dei puntatori.

11.5.1 Caratteristiche Principali

- **Inizializzazione obbligatoria:** Deve essere inizializzata subito: `int& r = x;`
- **Immutabilità:** Non può essere modificata per puntare a un altro oggetto dopo la definizione.
- **Sicurezza:** Non possono essere mai non inizializzate e non hanno mai il valore `nullptr`.
- **Efficienza:** Possono risiedere nei registri della CPU e non occupano sempre memoria fisica. L'operatore `sizeof()` restituisce la dimensione del tipo a cui la referenza punta.
- **Sintassi:** Vengono utilizzate come variabili normali, senza necessità di dereferenziazione.
- **Limitazione con gli Array:** Le referenze non possono essere usate per scorrere gli array tramite aritmetica (incremento/decremento). Per questo scopo, i **puntatori** rimangono indispensabili.

11.5.2 Confronto: Value vs Reference

Caratteristica	Call by Value	Call by Reference
Copia dei dati	Sì (costoso per oggetti grandi)	No (passa l'alias)
Modifica originale	No	Sì
Sintassi	`void f(int a)`	`void f(int& a)`

11.5.3 Best Practices

L'utilizzo delle referenze riduce drasticamente i rischi di errori rispetto ai puntatori. Tuttavia, i puntatori rimangono necessari in casi specifici:

- **Gestione degli Array:** Un array "decade"naturalmente in un puntatore al suo primo elemento (*pointer decay*), rendendo i puntatori lo strumento standard per iterare o manipolare blocchi di memoria contigui.
- **Programmazione Low-level:** Necessaria per l'interazione diretta con l'hardware o allocazione dinamica manuale (`new`/`delete`).

```c
// 1. Usa const per oggetti grandi in sola lettura
// Ottimizza le prestazioni impedendo modifiche accidentali
int foo(const BigType& b);

// 2. Passa SEMPRE struct e classi per referenza
void process(MyClass& obj);

// 3. ERRORE DA EVITARE: mai tornare referenze locali
int& func() {
    int x = 10;
    return x; // PERICOLO: x viene distrutta all'uscita!
} // La referenza diventerebbe "dangling" (punta al vuoto)
```

11.6 Speicherplatzbedarf von Objekten

Eine Unterklasse braucht immer mindestens so viel speicher wie eine Oberklasse.

11.6.1 Zuweisungen

Zuweisungen von Oberklasse zu Unterklasse sind in der regel möglich da alle geerbten Attribute ja vorhanden sind. Umgekehrt geht das allerdings nicht, da der Speicher dafür nicht initialisiert ist.

11.6.2 Slicing Problem

- **Definizione:** Si verifica quando un oggetto di una sottoclasse viene passato **per valore** a una funzione che si aspetta la superclasse.
  - Viene invocato il *Copy-Constructor* della superclasse.
  - Solo la parte "base"dell'oggetto viene copiata; le estensioni della sottoclasse vengono perse (slicing).
- **Conseguenza:** All'interno della funzione, l'oggetto si comporta esclusivamente come un'istanza della classe base, perdendo ogni comportamento polimorfico.
- **Soluzione:** Utilizzare sempre il passaggio **per referenza** (`const T&`) o tramite puntatori. Questo preserva l'integrità dell'oggetto originale e permette il corretto funzionamento dei metodi `virtual`.

11.7 Type Casting

	Beispiele	Wissenswertes	Anwendungsbereiche

implizit: kein Cast	1. signed char a = 1;
	int b = a;
	
	2. int a = 1;
	char b = a; // ggf. mit Warnung
	
	3. Bird b;
	Animal a = b;// Upcast: a ist ein Animal
			• Numerische Typen u. bool (ggf. mit Warnung, falls Wertebereich möglicherweise nicht ausreicht)
			• Objektinstanzen einer Unterklasse in einer Variablen vom Typ der Oberklasse speichern (Upcast).
			• Pointertypen, deren Umkonvertierung «ungefährlich» ist – die genauen Regeln sind ziemlich kompliziert.

static	1. int a = 1 ;
	char b = static_cast<char>(a);
	2. Bird* b = new Bird;
	Animal* a = b; // geht implizit!
	Bird* c = static_cast<Bird*>(a);
	/* Braucht einen down-Cast , weil nicht jedes Animal ein Bird ist!*/
		Per convertire un tipo intero più grande in uno più piccolo, il compilatore semplicemente "taglia" i bit in eccesso, mantenendo solo quelli meno significativi (LSB).	• Alles, was implizit möglich ist, unterdrückt dabei etwaige Warnungen.
			• Pointer- und Referenz-Typen auf Instanzen von Ober- und Unterklassen in beide Richtungen umwandeln: Upcast und Downcast (**ohne Typprüfung** zur Laufzeit, Auswertung ausschliesslich zur **Compilezeit**)

dynamic	Animal* a = new Ant; Bird* b = dynamic_cast<Bird*>(a); /* Der Dynamic Cast wird immer zur Laufzeit ausgewertet, dh: b ==nullptr */	Unterstützt ausserdem einen Upcast für Pointer- und Referenztypen auf nichtpolymorphe Klassen (normalerweise machen wir dies aber implizit oder verwenden einen static_cast!) Falls der dynamic_cast **fehlschlägt**, liefert er bei Pointer-Typen den Wert **nullptr**, bei Referenzen wird eine **std::bad_cast-Exception** geworfen	• Pointer- und ReferenzTypen auf Instanzen von **polymorphen** Ober- und Unterklassen in beide Richtungen umwandeln: Upcast und Downcast (**mit** Typprüfung per RTTI zur Laufzeit)

const	const int val = 10;
	const int *ptr = &val;
	int *ptr1 = const_cast <int *>(ptr);
			verändert die `constness` und/oder `volatile-ness` des Ziels eines Pointer oder Referenz Typs.

reinterpret	Animal* a = new Ant; uint8_t* b = reinterpret_cast<uint8_t*>(a);	Achtung; erzeugt plattformspezifisches Verhalten	• Konvertiert zwischen beliebigen Pointer-Typen sowie zwischen beliebigen Pointer und int-Typen **ohne Typprüfung**. • Kann auch mit Referenz-Typen umgehen.

Die C Syntax für Casting sollte nicht mehr verwendet werden da meistens nicht optimales verhalten.

11.8 Templates

- **Indipendenza dal tipo:** Consentono di scrivere codice generico utilizzando dei **placeholder** (segnaposti) al posto di tipi specifici.
- **Sicurezza dei tipi (Type Safety):** A differenza del C (`void*`), i template garantiscono un controllo dei tipi tramite *early binding* in fase di compilazione.

- **Parametri di istanziazione:** Oltre ai tipi, è possibile passare anche delle **costanti** come parametri del template
- **Versatilità:** Possono essere applicati sia a **classi** che a **funzioni**.
- **Valutazione a tempo di compilazione:** Il compilatore genera il codice effettivo durante la fase di **compile-time**.
- **Struttura e File:** Richiedono dichiarazione, definizione e parametri; solitamente vengono definiti interamente negli **header file** (.h / .hpp).
- **Generazione del codice:** Il compilatore crea una copia separata della funzione o classe per ogni **diverso tipo** utilizzato.
- **Zero Dead Code:** Se un template non viene mai utilizzato nel codice, non viene generata alcuna istruzione nel binario finale
- **Footprint di memoria e Code Bloat:** L'istanziazione per molteplici tipi, oppure l'uso eccessivo della keyword `inline` per definizioni lunghe, può causare *code bloat*.

11.8.1 Implementation

Die Typendefinitionen sind dann durch das T zu ersetzen.

```
// H-File
template<typename T>
void swap(T& a, T& b); // deklaration

template<typename T> // definition (immer .h)
void swap(T& a, T& b) { // REFERENCES for parameter
    T temp{b};
    b = a;
    a = temp;
}
```

As the dimension of the paramters is unknow ALWAYS use references

11.8.2 Syntax

```
// Template-Definition fuer Funktionsdeklaration:
template<typename type1, typename type2, ...>
returntype name(type1 param1, type2 param2 , ...);
// Inline goes before returntype

// Template-Definition fuer Funktionsdefinition:
template<typename type1, typename type2, ...>
returntype name(type1 param1, type2 param2 , ...) { ... }

// Klassentemplate:
// Template-Definition fuer Klassendeklaration:
template<typename type1, typename type2, ...>
class Classname{ ... };

// Template-Definition fuer Methode einer Klasse:
template<typename T1, typename T2, ...>
returntype Class-name<T1, T2, ...>::method-name(type1 param1,
     type2 param2 , ... ) { ... };
```

11.8.3 Instanziierung

L'istanziazione è il processo con cui il compilatore genera codice macchina concreto

- **Implicita:** compilatore deduce il tipo dai parametri passati **Nota bene:** il tipo di ritorno non viene mai considerato per la deduzione
- **Esplicita:** Il tipo viene specificato manualmente tra parentesi angolari

Klassentemplates (<> obbligatori):
tipo sempre specificato per permettere al compilatore di determinare l'ingombro in memoria.

```
Stack<int, 10> s1; // Istanzia uno Stack di 10 interi
```

Funktionstemplates (<> opzionali se deducibili):
Se il tipo è chiaro dai parametri, le parentesi angolari possono essere omesse.

```
int i = minimum(iF, size); // Deduzione implicita del tipo int
int i = minimum<int>(iF, size); // Qualificazione esplicita
```

11.8.4 Overloading

I template di funzione possono essere sovraccaricati (overloaded) con altri template o con funzioni "normali". In caso di chiamate omonime, il compilatore valuta le funzioni compatibili, istanzia quelle dei template, le aggiunge alle funzioni normali disponibili e infine seleziona la variante che offre il *match* migliore con i tipi dei parametri passati.

11.8.5 Kompilierung und Code-Organisation

- **Header-Only:** il compilatore deve 'vedere' la definizione per ogni traduzione. Lo svantaggio principale è l'**aumento dei tempi di compilazione**
- **Explicit Instantiation (.cpp):** Per ridurre i tempi di compilazione, è possibile spostare la definizione in un file `.cpp` e forzare l'istanziazione per tipi specifici (es. `template class Stack<int>;`).

12 Exceptions

Exceptions werden zur Behandlung von Fehlern verwendet. Eine Exception ist eine Instanz eines beliebigen Datentyps, welche Fehlerinformation transportiert. Eine Exception wird bei Fehlerauftritt `geworfen` (throw), und zur Behandlung `gefangen` (catch).

```
try {
    // dangerous Code with throw
} catch (someType someName) {
    // some error handling.
}
// normal execution continues here
```

Diese Syntax ist so in der Praxis wenig aufzufinden da das Fehler werfen meist in einer Unterfunktion gemacht wird. Eine geworfene Exception wird dann mit code um die Instanziierung gehandhabt. Dies erlaubt es Fehler von Unterfunktionen je nach Situation anders zu handhaben:

```
#include <iostream>

int divide(int dividend, int divisor) {
    if (divisor == 0) throw "Division by zero!";
    return dividend/divisor;
}
int foo(int a, int b) {
    std::cout << "Entering foo()" << std::endl;
    int result = divide(a, b); // can throw exception
    std::cout << "Leaving foo()" << std::endl;
    return result;
}

int main() {
    try {
        std::cout << foo(7,0) << std::endl;
    } catch (const char* e) {
        std::clog << "Not optimal but ok, " << e << std::endl;
    }// just writes the Error to the log

    // continue Program Function
    std::clog << "Continue Programm" << std::endl;

    try {
        std::cout << foo(12,0) << std::endl;
    } catch (const char* e) {
        std::cerr << "Critical error: " << e << std::endl;
        std::cerr << "Program terminating" << std::endl;
        exit(-1);
    }
    std::clog << "finish Programm" << std::endl;
    // continue Program Function
}
```

Cout:

- Entering foo()
- Not optimal but ok, Division by zero!
- Continue Programm
- Entering foo()
- Critical error: Division by zero!
- Program terminating

12.1 Mehrere Catch Blöcke

Unterschiedliche Exceptions können mit mehreren seriellen Catch Blöcken erreicht werden. Wenn der letzte Catch block mit `...` aufgerufen wird, werden alle Typen Exceptions gehandelt.

```
int main() {
    int x;
    std::cin >> x;
    try {
        a(x); // can throw various types of exceptions
    } catch (const char* e) {// handles only Type const char*
        std::clog << e << std::endl;
    } catch (int i) {// handles only Type int
        std::cerr << i << std::endl;
    } catch (...) {// handles whatever...
        std::cerr << "other exception occured" << std::endl;
    }
return 0;
```

12.2 Fehlender catch block

Falls kein Passender catch Block gefunden werden kann wird die Funktion std::terminate() aufgerufen. Diese ruft den registrierten **std::terminate_handler** (Funktionspointer). Defaultmässig ist **std::abort()** hinterlegt welche das Programm beendet ohne Destruktoren zu rufen (crash). Falls dieses Verhalten angepasst werden soll, kann man das mit `set_terminate(std::terminate_handler f)`.

12.3 leerer Catch block

straight to hell

12.4 noexcept

Um zu signalisieren, ob eine Funktion eine Exception auslösen kann oder nicht, kann der specifier **`noexept`** verwendet werden. Dafür ist aber **der Entwickler zuständig**. Der Compiler kann **nicht** prüfen, ob die Funktion tatsächlich eine Exception wirft oder nicht.

12.5 <stdexcept>

stdexcept ist dazu da um einen einfachen Austausch von Fehlerinformationen vorzunehmen. Dafür sind diverse standardisierte Fehlertypen als Klassen definiert. Z.b.:

`std::logic_error` **std::out_of_range** **std::overflow_error**

Für Details siehe :https://en.cppreference.com/w/cpp/header/stdexcept

13 STD Library

13.1 array

- **Definizione:** `std::array<T, size>` è un contenitore a dimensione fissa implementato come classe template.
- **Dimensione:** A differenza dei classici array in stile C, `std::array` conosce la propria dimensione tramite la funzione membro `.size()`.
- **Accesso ai dati:** È possibile accedere agli elementi utilizzando la classica notazione a parentesi quadre `[]`.
- **Sicurezza:** Il metodo `.at()` fornisce un accesso sicuro con controllo dei limiti, lanciando un'eccezione `std::out_of_range` in caso di accesso non valido.
- **Integrazione:** Funziona in modo ottimale con le moderne funzionalità del C++, inclusi i cicli *range-based for*.

13.2 vector

std:vector ist ein Klassentemplate welches die Kapazität selbständig bei Bedarf erweitert.

```cpp
#include <vector>
std::vector<dt> v;// Template instanzieren mit <datentyp> (dt)
v.push_back("Value");// Value anfuegen
v.erase("iterator");// Elemente loeschen(Iterator=fancy Pointer)
v.begin();// returned ein Pointer auf das erste element
v.end();// returnd ein Pointer HINTER das letzte Element

for (const dt s : v) { // Iteriert ueber Elemente vom Vector
    doSomething(s);} // do something mit element

for (std::vector<dt>::iterator it=v.begin();it!=v.end();it++) {
    doSomething(s);} // do something mit element, altes for
```

14 Makefiles

Makefiles sollten generell das Umsetzten des Codes in Maschinencode vereinfachen. Es ist eine `Skriptingsprache` welche grob nach dem Muster `Erzeugnis/target : Abhängigkeiten/Dependency` folgt. Dann folgt indentiert der Befehl, um dieses Erzeugnis zu erzeugen.
Wenn ein Erzeugnis keine Datei zurückgibt, muss dieser als `.PHONY` markiert werden. Das verhindert, dass eine Datei mit demselben Namen das Ausführen verhindert.
Am besten sieht man das an einem Beispiel:

```makefile
all : stuff                 # erzegt die Definition fuer make all

project : main.o lib.o      # Projekt linken
  clang++ -Wall -o vector Vector3D.o main.o

lib.o : lib.cpp             # lib.cpp compilen
  clang++ -Wall -c lib.cpp

main.o : main.cpp           # main.cpp compilen
  clang++ -Wall -c main.cpp

clean :                     # Projekt cleanup
  rm -f project.exe lib.o main.o

.PHONY : clean all          # Mariert die Targets "clean" und "all" als "PHONY" -> Lesbarkeit
```

Der Befehl `make all` baut nun das Projekt.
Sind die o Dateien noch aktuell werden diese nicht erneut kompiliert. Das Projektverzeichnis kann einfach durch `make clean` aufgeräumt werden.

14.1 Platzhalter

In einem Makefile können auch Platzhalter verwendet werden:

- **$ @** Dateiname des Targets
- **$ <** Dateiname der ersten Dependency des aktuellen Targets
- **$ ∧** Dateiname aller Dependencies des aktuellen Targets durch Leerzeichen getrennt

Mit diesen Mitteln kann ein Professionelleres Makefile erstellt werden, welches relativ universell eingesetzt werden kann:

```makefile
CXX = clang++               # Verwendeter Compiler
CXXFLAGS = -Wall -c         # compilerflags
LDFLAGS = -o                # loader flags
BIN = main                  # binary output / Name des .exe file
OBJS = drink.o drinktest.o  # objectfiles

all: $(BIN)                 # Make all befehl

%.o: %.cpp                  # cpp Files zu o dateien compilen
  $(CXX) $(CXXFLAGS) $<

$(BIN): $(OBJS)             # Linken zu exe file
  $(CXX) $(LDFLAGS) $@ $^

run: $(BIN)                 # Projekt ausfuehren
  ./$(BIN)

clean:                      # o. Dateien entfernen
  rm -f $(OBJS) $(BIN)

.PHONY: clean all           # Markiert clean & all als PHONY
```

15 Anhang

15.1 Styleguide

- **Variablen, Konstanten**
  - mit Kleinbuchstaben beginnen
  - erster Buchstaben von zusammengesetzten Wörtern ist gross (mixed case)
  - keine Underscores

  Beispiele : counter, maxSpeed
- **Funktionen**
  - mit Kleinbuchstaben beginnen
  - erster Buchstaben von zusammengesetzten Wörtern ist gross (mixed case)
  - Namen beschreiben Tätigkeiten
  - keine Underscores

  Beispiele : getCount(), init(), setMaxSpeed()
- **Klassen, Strukturen, Enums**
  - mit Grossbuchstaben beginnen
  - erster Buchstaben von zusammengesetzten Wörtern ist gross (mixed case)
  - keine Underscores
  - Namen beschreiben Dinge

  Beispiele : MotorController, Queue, Color

15.2 Code beispiele

15.2.1 String formatting

Utilizza gli stream di output per costruire stringhe in memoria. È sicuro perché gestisce la memoria dinamicamente.

```cpp
#include <sstream>
#include <iomanip>

std::string formatTime(int h, int m) {
    std::ostringstream buf;
    buf << std::setfill('0') << std::setw(2) << h
        << ":" << std::setw(2) << m;
    return buf.str();
}
```

Rappresenta lo standard moderno. Combina la sintassi concisa di `printf` con la sicurezza degli stream.

```cpp
#include <format>

std::string formatTime(int h, int m) {
    // :02 significa 2 cifre con riempimento zero
    return std::format("{:02}:{:02}", h, m);
}
```

15.2.2 Ctor un dtor

```cpp
class TString {
public:
    TString();                          // 1: Default Ctor
    TString(const TString& s);         // 2: Copy Ctor
    TString(const char* p);            // 3: Ctor da stringa
    explicit TString(int i);           // 4: Ctor esplicito da int
    // ...
};

TString s1 = "Hoi";     // 3
TString s2;             // 1
TString s21 = s1;       // 2
s2 = s1;                // Nessun Ctor (Assignment operator)
TString& s3 = s21;      // Nessun Ctor (Riferimento)
TString s4[2];          // 1, 1 (Array di 2 elementi)
TString* p1 = &s21;     // Nessun Ctor (Puntatore)
TString* p2 = new TString{"Hallo"}; // 3 (Allocazione su heap)
TString* p3 = new TString[3];       // 1, 1, 1 (Array di 3 su heap)
TString s5[3] = {"Hoi", TString{}, s2}; // 3, 1, 2

// Esempio per il Ctor 4 (explicit)
TString s13 = TString{13};              // 4
```

15.3 Ritorno per Riferimento e Method Chaining

- **Meccanismo:** Una funzione restituisce un riferimento all'oggetto stesso utilizzando `return *this;`. Il tipo di ritorno deve essere una referenza (es. `Complex&`).
- **Kaskadierung (Cascading):** Permette di chiamare più metodi in sequenza sulla stessa istanza in un'unica riga: `obj.funz1().funz2();`.
  Tramite ritorno *by-value* funz2 verrebbe eseguita su un oggetto *tmp*
- **Efficienza:** A differenza del ritorno per valore, non viene creata alcuna copia dei dati (nessun oggetto temporaneo *tmp*), risparmiando risorse
- **Persistenza:** Le modifiche effettuate nella catena agiscono direttamente sull'oggetto originale e non su una sua copia destinata alla distruzione.

```cpp
// .h
class Complex
{
  public:
    Complex(const Complex& c);   // Copy constructor
    Complex& add(const Complex& c); // adds c to the current object
};
// complex.cpp
Complex& Complex::add(const Complex& c) {
    this->re += c.re;
    this->im += c.im;
    return *this;
}

// main.cpp
    Complex c3 = c1.add(c0).add(c2);
    // c1 viene aggiornato correttamente
    // c3 viene creato tramite copy ctor
```

15.4 Vergleich von Gleitkommazahlen (double)

Beim Vergleich von `double`-Werten sollte aufgrund von Rundungsfehlern **niemals der Operator == direkt verwendet werden**.
Stattdessen wird geprüft, ob die absolute Differenz zwischen zwei Werten kleiner als eine definierte Toleranz (Epsilon) ist.

```cpp
#include  <cmath>

bool areEqual(double a, double b) {
    static constexpr double epsilon = 1e-9;
    return fabs(a - b) < epsilon;}
```

15.5 Parametri Template Non-Tipo (Static Allocation)

I template accettano valori costanti (es. `std::size_t N`) valutati a **compile-time**. Questo è l'unico modo per definire la dimensione di array statici (`T data[N]`) mantenendo la flessibilità del codice.
In C++, il compilatore deve conoscere l'ingombro esatto di un oggetto nello **Stack** per generare istruzioni macchina con *offset* fissi. Se `N` fosse una variabile, il compilatore non saprebbe quanto spazio riservare automaticamente e l'allocazione fallirebbe.

15.5.1 Allocazione Statica vs Dinamica

Senza la risoluzione a tempo di compilazione, il compilatore non potrebbe riservare lo spazio nello **Stack**, obbligando all'uso della memoria **Heap** (più lenta).

Caratteristica	Via Template (Statico)	Via Costruttore (Dinamico)
Vincolo Size	Costante a compile-time	Variabile a run-time
Memoria	Stack (Veloce, fissa)	Heap (Flessibile, lenta)
Gestione	Automatica (No memory leaks)	Manuale (`new`/`delete`)
Effetto Tipo	`Class<10>` ≠ `Class<20>`	Il tipo rimane identico

15.5.2 Esempio e Default

È possibile impostare un valore di default per rendere il parametro opzionale.

```cpp
template<typename T, std::size_t N = 100>
class Container {
    T data[N]; // Legale solo perche N e noto al compilatore
};

Container<int, 20> c1; // Alloca spazio per 20 int sullo Stack
Container<int> c2;     // Usa il default di 100
```

15.6 Inline Functions e Templates

- **Inlining Implicito:** Tutte le funzioni (template e non) definite direttamente all'interno della dichiarazione di una classe (*in-class*) sono considerate `inline` automaticamente dal compilatore.
- **Inlining Esplicito:** Le funzioni template definite al di fuori della classe (*out-of-class*), anche se collocate nello stesso file header, **non** sono `inline` di default. Per renderle tali, la keyword `inline` deve essere inserita tra il tag `template` e il tipo di ritorno.
- **Vantaggi e Limiti:** L'inlining elimina l'overhead della chiamata a funzione (*zero overhead*), ma è sconsigliato per funzioni ricorsive o molto complesse.